

Thema: Konzeptionieren eines Coasterbots

Endbericht



Team: Coasterbot

Studierender	Matrikelnummer
Dennis Borutta, B. Eng	857962
Stefan Etringer, B. Eng	865769
Gereon Such, M. Sc.	867070

Betreuer: Dr.-Ing. Winfried Tiedge

Abgabedatum: 07.11.2026

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	25.09.2026	Borutta	Erster Entwurf.

Inhaltsverzeichnis

1	Einleitung	4
2	Projektumsetzung	4
2.1	Projektziel (Dennis Borutta)	4
2.2	Lösungsweg: Mechanischer Aufbau (Dennis Borutta)	4
2.3	Lösungsweg: Algorithmik und Simulation (Stefan Etringer)	4
2.4	Lösungsweg: Elektronik und Abstraktion (Gereon Such)	4
2.5	Projektergebnis (Dennis Borutta)	9
2.6	Zeitplan Soll/Ist (???)	9
2.7	Kosten- und Ressourcenplan Soll/Ist (Gereon Such)	9
2.7.1	Personal	9
2.7.2	Material	10
2.8	Nutzen und Weiterentwicklung des Ergebnis (???)	11
3	Projektablauf	11
3.1	Teambildung und Rollenverteilung (Dennis Borutta)	11
3.2	Zusammenarbeit im Projekt (Stefan Etringer)	11
3.3	Kommunikation im Projekt (Gereon Such)	12
3.4	Bewertung der Zusammenarbeit	13
3.5	Bewertung des Projektergebnis	14

1 Einleitung

2 Projektumsetzung

Planumfang ca. 9 Seiten Davon: Dennis 2 Seiten, Stefan 2 Seiten, Gereon 2 Seiten Offen: 3 Seiten

2.1 Projektziel (Dennis Borutta)

Inhalt ca 0,5 Seiten

2.2 Lösungsweg: Mechanischer Aufbau (Dennis Borutta)

Inhalt ca 1 Seite

- Erarbeitung des Lösungsweges: Benchmarking, Abgrenzung Ihres Vorgehens und der Wahl der Mittel von alternativen Lösungsmöglichkeiten, Begründung Ihres Ansatzes

2.3 Lösungsweg: Algorithmik und Simulation (Stefan Etringer)

Inhalt ca 2 Seiten

- Erarbeitung des Lösungsweges: Benchmarking, Abgrenzung Ihres Vorgehens und der Wahl der Mittel von alternativen Lösungsmöglichkeiten, Begründung Ihres Ansatzes

2.4 Lösungsweg: Elektronik und Abstraktion (Gereon Such)

Inhalt ca 2 Seiten

- Erarbeitung des Lösungsweges: Benchmarking, Abgrenzung Ihres Vorgehens und der Wahl der Mittel von alternativen Lösungsmöglichkeiten, Begründung Ihres Ansatzes

Ausgangslage und Leitlinien

Die im Pflichtenheft definierten Hardwarefunktionen mussten durch entsprechende Sensorik, Aktorik und Rechenleistung abgedeckt werden:

- HW-CPU-001..004 Rechenleistung für Sensordatenverarbeitung und Navigation
- HW-SEN-001..005 Hindernis und Tischkantenerkennung
- HW-SEN-006/007 Positionierung und Orientierung
- HW-ACT-001..004 Bewegungssteuerung
- HW-ACT-005..007 Untersetzeraufnahme und -abgabe
- HW-PWR-* Energieversorgung

Ein weiteres Kriterium bei der Komponentenauswahl war schnelle und problemlose Verfügbarkeit in geringen Stückzahlen sowie ein geringer Preis. Da eine PCB Fertigung inklusive Versand einen enormen Zeitaufwand impliziert, z.B. JLCPCB ca. 4 Wochen, wurden alle Komponenten als Development-Boards beschafft und diese auf Lochrasterplatine sowie mit Steckverbindern auf Pinheadern verbunden. Jede Komponente wurde iterativ in Betrieb genommen und die Funktionalität sowie Eignung einzeln verifiziert. Dabei ergeben sich in einzelnen Fällen Schwierigkeiten, die Anpassungen erforderten, dies soll im Anschluss an die ursprüngliche Auswahl der Komponenten erläutert werden.

Benchmarking und Auswahl der Komponenten

Die Auswahl der Komponenten wurde in Phase M3 (Alternativenbeschreibung) diskutiert, hier wurden für jede Anforderung Lösungsvarianten identifiziert und eine ausgewählt. Tabelle 1 fasst die wesentlichen Entscheidungen zusammen.

Tabelle 1: Betrachtete Alternativen und gewählte Lösung der Elektronik.

Funktion	Alternativen	Gewählt	Begründung
Controller	Arduino Uno, ESP32, Raspberry Pi Pico2	Raspberry Pi Pico 2 (RP2350)	Uno nicht performant genug. Pico2 nimmt weniger Leistung auf als ESP32 und hat mit PIO Interface Möglichkeit zeitkritische Signale zu generieren.
Hinderniserkennung	Ultraschall HC-SR04, Kamera OV7670, Bumper, IR-Sensor	HC-SR04 in Fahrtrichtung	Unfälle auf dem Tisch sind zu wahrscheinlich mit Bumper Sensor. HC-SR04 bietet einfache und günstige Distanzmessung ohne Bildverarbeitung. Die Kamera wurde wegen des Verarbeitungsaufwands auf dem Mikrocontroller zurückgestellt.
Tischkanten-erkennung	IR-Reflexsensor, Kamera	4 IR-Sensormodule (je Ecke)	Kamera zurückgestellt, daher alternativlos.
Lage und Odometrie	IMU, Radencoder, Kamera	MPU-6050 (I ² C) und LM393-Gabellichtschranken	Die IMU (Inertialsystem) liefert die Drehung, die Encoder die gefahrene Strecke. Die Radencoder wurden während der Testphase ergänzt (siehe unten).
Untersetzermechanik	Micro-Servo, Schrittmotor	2 Micro-Servos (MG90)	Positionsgeregelt, nur ein Signalpin, aufgrund hohen Widerstands des PETG Aufbaus mussten Metallgewinde gewählt werden.
Antrieb	TT-Motoren mit H-Brücke	4 TT-Motoren (1:48), 2× TB6612FNG	Günstig und als Set mit Rädern verfügbar. Der Regler TB6612FNG wurde aufgrund geringerer Verlustleistung als der L298N gewählt.
Energieversorgung	9-V-Block mit Linearregler 78x05; Li-Ion-Akku mit DC/DC-Wandler	18650-Li-Ion-Akku (3.7 V) mit Step-Up-Wandler auf 5 V	Die gewählten Motoren bedürfen recht hoher Ströme, dieser kann nicht durch Linearregler geliefert werden. Folglich Motorspannung VM direkt am Akku, Steuerspannung VCC über Step-Up-Wandler.
Bedienung und Status	RGB-LED oder diskrete LEDs; Taster	RGB-LED, Taster, Hauptschalter	Eine RGB-LED kann alle Betriebszustände über Farbe und Blinkmuster darstellen.

Elektrischer Aufbau

Die Schaltung wurde in der Open Source Software KiCad erstellt (Abbildung 1) und mit der Software DIYLC auf ein Lochrasterplattenlayout im Raster 30×24 übertragen. Externe Komponenten werden über Pin Header angeschlossen. Die Platine trägt den Mikrocontroller, den Step-Up-Wandler, die Motortreiber, das Inertialsystem, die RGB-LED, Glättungskondensatoren für die Versorgungsspannungen sowie Pulldown Widerstand und Endprellung für den Taster. Der Aufbau ist in der Aufbauanleitung (M4) detailliert dokumentiert.

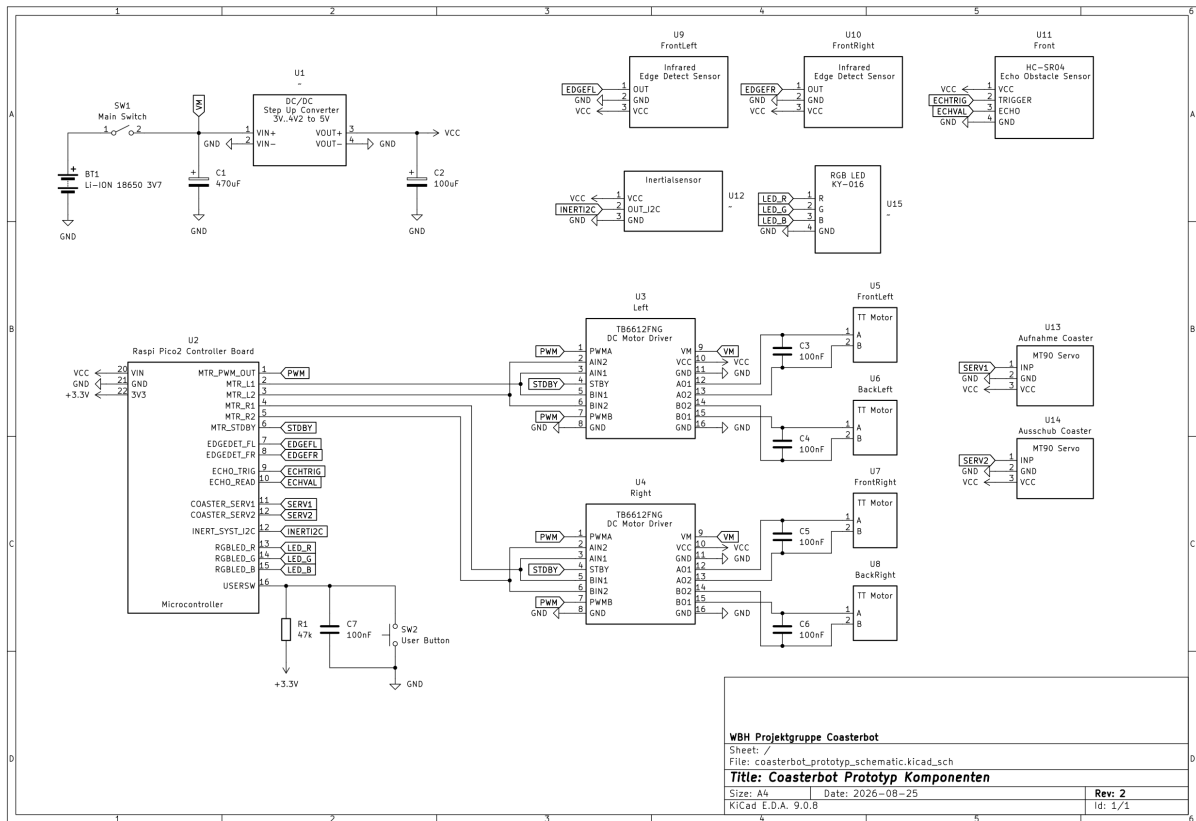


Abbildung 1: Gesamtschaltung der Elektronik (aus der Aufbauanleitung).

Beide Motoren einer Seite sind durch je einen Motortreiber geregelt, wobei sich beide Motortreiber das Standby- sowie PWM-Signal teilen. Dadurch drehen alle Motoren stets betragsmäßig gleich schnell. In der Folge sind nur Geradeausfahrt vorwärts/rückwärts als auch bei entgegengesetzter Drehrichtung Kurven auf der Stelle möglich. Diese Vereinfachung kann für eine zukünftige Ausbaustufe eventuell überdacht werden, da die Positionserkennung durch Odometrie (Zählen der Rad-Rotationen) bei Rotation auf der Stelle verhältnismäßig ungenau ausfällt. Dies ergab sich in den Inbetriebnahmetests und wurde durch das Inertialsystem kompensiert.

Hardwareabstraktion

Die zentrale Schnittstelle ist die abstrakte C++-Klasse `RobotHAL`, die in der Simulation parallel zur Entwicklung der Elektronik definiert wurde. Sie beschreibt die Fähigkeiten des Roboters (Geschwindigkeit je Seite, Objektdistanz nach vorne, Kantensensor, Drehrate, Encoderwinkel, Zeit, Servosteuerung und LED), und referenziert weder die Webots Implementierung noch die eigentlichen Hardwareimplementierungen. Diese abstrakte Basisklasse wird dann in der `HardwareImplementationHAL` implementiert und bindet dann die Controller der einzelnen Komponenten ein. Diese Controller sind im Projekt

Source Code als Template Klassen für die Hardwareparameter auf dem Arduino Framework implementiert. In der Hardwareabstraktion werden dann alle Controller mit den entsprechenden Pins aus einer Projektkonfigurationsdatei instanziiert und kombiniert.

Alle Komponenten sind *non-blocking* implementiert. Wenn z.B. die Motorsteuerung einer Seite sich derzeit in Vorwärtsdrehung befindet, dann durch die Steuerung aber ein Richtungswechsel ausgelöst wird, so muss zunächst aktiv abgebremst werden und erst wenn dieser Bremsvorgang beendet ist darf der Motor wieder in Rückwärtsrichtung angesteuert werden. Solche zeitabhängigen Vorgänge können natürlich durch `delay/sleep`-Befehle gelöst werden, in dieser Zeit erfolgt aber auch keine Sensorauswertung oder weitere Parametrierungen. Der Non-blocking Ansatz implementiert in den Komponenten jeweils eine interne Statemachine, die nach variablen Zeitintervallen einen Zustandsübergang durchführt und die Ausgangssignale anpasst oder nur in fixen Zeitintervallen Sensorwerte wiederum ausliest. Alle Aktionen finden in der Anwendungslogik weiterhin seriell statt, aber in geplanten Intervallen in `step` oder `update` Methoden, die in jedem Hauptschleifenzyklus durchlaufen werden und meistens direkt zurückspringen (Early Exit).

Anpassungen während der Umsetzung

Der ursprüngliche Ansatz für das Positionierungssystem war eine Integration des Beschleunigungssensors um einen Geschwindigkeitswert zu erhalten, Da die TT Motoren eine solche Information schlicht nicht bereitstellen können. Dieser Geschwindigkeitswert sollte wiederum integriert werden um eine absolute Positionierung zu erhalten.

Während der Hardwaretests zeigte sich, dass trotz Bias-Kalibrierung, Lagekorrektur und Stillstandserkennung in Verbindung mit der Motorsteuerung die erreichbare Genauigkeit für ein Positionssystem außerhalb akzeptabler Toleranz lag: Durch Testfahrten immer wieder vorwärts und rückwärts summierte sich ein Fehler auf die aktuelle Position von einigen Dezimetern mit steigender Tendenz. In der Folge wurde die Motorhalterung modifiziert und an der dem Rad abgewandten Seite zweier TT Motoren eine Gabellichtschranke mit 20-Loch Drehscheibe und LM393 Komparator angebracht. Durch diese Kodierung ist die Änderung des Radwinkels mit $\frac{1}{40}$ des Umfangs bestimmbar, bei Raddurchmesser von 66 Millimeter ergibt sich bei einem *Tick* am Sensor(Wert-Wechsel) eine Fahrtstrecke von $66 \times \pi/40 = 5.2$ Millimeter. Dies ist für eine gemessene Strecke zwar noch akzeptabel, aber bei Rotation wiederum zu ungenau um einen echten Drehwinkel für ein Navigationssystem zu bestimmen, weshalb der Drehwinkel weiterhin durch das IMU-System bezogen wird, wo Rotationswinkel weniger fehlerbehaftet sind als lineare Beschleunigungen. Bei der Implementierung der Tick-Erkennung wurde zusätzlich noch mit einem Hardwareinterrupt gearbeitet und konsekutive Ticks aufsummiert um sicherzustellen, dass nicht durch zu geringe Hauptschleifenfrequenz Ticks verloren gehen. Zudem wurden beide dieser Odometriecontroller mit der Motorsteuerung gekoppelt, sodass die Drehrichtung des jeweiligen Rades berücksichtigt wurde.

Weitergehend wurde festgestellt, dass eine Drehbewegung auf der Stelle ein deutlich höheres Drehmoment im Gegensatz zu Geradeausfahrt bedarf. Dies wurde in der Motorsteuerung berücksichtigt, allerdings ist der Schlupf der Räder auf einer Tischoberfläche hier sehr hoch. Vermutlich ist also in der Folge eine Navigation mit *sanften Kurven* doch zweckmäßiger. Dies sollte für eine weitere Ausbaustufe wie bereits erwähnt in Betracht gezogen werden.

Derzeit noch nicht umgesetzt sind die Akkuüberwachung (HW-PWR-002/003) sowie eine Rückkopplung von Odometriesystem zu Motorsteuerung um eine tatsächliche Geschwindigkeitsregelung auf $\frac{m}{s}$ zu erreichen. Die Ansteuerung der Motoren erfolgt über das PWM Signal, das lediglich von 0 für minimalem bis 255 für maximales Drehmoment parametrisiert werden kann, die tatsächliche Drehgeschwindigkeit ist aber dann wiederum von der Motorspannung VM abhängig.

2.5 Projektergebnis (Dennis Borutta)

Inhalt ca 0,5 Seiten

2.6 Zeitplan Soll/Ist (???)

Inhalt ca 1 Seite

2.7 Kosten- und Ressourcenplan Soll/Ist (Gereon Such)

Inhalt ca 1 Seite

Im Kostenplan aus M0 wurde die Arbeitszeit wie in einem Unternehmen mit einem Selbstkostensatz von 89€/h angesetzt. Werkzeuge wurden als gegeben vorausgesetzt, Software sollte ausschließlich Open Source verwendet werden. Dies konnte so gehalten werden.

2.7.1 Personal

Für das Projekt waren 580 Personenstunden (ca. 15 h pro Woche pro Person auf Projektlaufzeit) was wiederum $580 \times 89\text{€} = 51620\text{€}$ geplant. In der Planungsphase wurde diese Wochenstundenzahl unterschritten, durch die notwendigen nachträglichen Änderungen bei der Identifizierung der Probleme mit dem Positionierungssystems wurde die Wochenstundenzahl überschritten. Aus Teambewertung ist dies für ein exploratives Projekt aber akzeptabel, da das konkrete Problem kaum voraussehbar war und sich während der Implementierung ergab.

2.7.2 Material

Die Ist-Kosten der Komponenten wurden aus den tatsächlichen Kleinpackungspreisen im Einzelhandel berechnet und sind Netto Beträge. Tabelle 2 listet die Komponenten nach Packungspreis, Stückzahl und resultierendem Projektpreis.

Tabelle 2: Ist-Kosten der Elektronik je Prototyp, berechnet aus den Packungspreisen.

Komponente	Packungspreis	Packung [Stk.]	verbaut [Stk.]	Anteil
Raspberry Pi Pico 2	13,69€	1	1	13,69€
TT-Motor mit Rad	12,99€	4	4	12,99€
Radencoder LM393	7,79€	5	2	3,12€
Motortreiber TB6612FNG	7,06€	2	2	7,06€
Micro-Servo	17,13€	10	2	3,43€
Li-Ion-Akku 18650	12,99€	4	1	3,25€
Inertialsensor MPU-6050	8,99€	3	1	3,00€
Verbinder	8,49€	120	30	2,12€
Ultraschallsensor HC-SR04	11,99€	8	1	1,50€
IR-Sensor Tischkante	9,59€	10	4	3,84€
RGB-LED	3,89€	5	1	0,78€
DC/DC-Wandler	5,96€	10	1	0,60€
Taster	4,99€	10	1	0,50€
Pfostenleiste	6,99€	50	2	0,28€
Motor-Kondensatoren 100 nF	9,99€	300	4	0,13€
Hauptschalter	9,59€	80	1	0,12€
Elkos	6,99€	145	2	0,10€
Summe Elektronik	159,11€			56,49€

Die angesetzten Kosten für das 3D-Druck Filament von 15 € können für den Druck nicht korrekt nachvollzogen werden, sind aber weiterhin ein realistischer Wert. Im Kostenplan aus M0 wurden Materialkosten von insgesamt $40 + 90 + 15 + 20 = 165\text{€}$ für Komponenten und Montage sowie 15 € für Filament angesetzt. In Tabelle 3 werden diese Werte gegen die tatsächlichen Werte ohne den in M0 angenommenen Risikopuffer verglichen werden. Die niedrigeren Materialkosten konnten durch kostengünstige Komponenten sowie Verzicht auf Kamera und eigene PCB Leiterplatte stark reduziert werden, ebenso wie durch den Bezug von Mehrfachverpackungen und Aufteilung der Komponenten in der Projektgruppe.

Tabelle 3: Projektkosten Soll/Ist je Prototyp.

Kostenart	Soll	Ist
Elektronik (Tabelle 2)	165 €	56,49 €
3D-Druck und Kleinteile	15 €	ca. 15 €
Material gesamt	207 €	ca. 72 €
Personal	51.620 €	51.620 €
Software und Werkzeuge	0 €	0 €
Gesamt	ca. 51.827 €	ca. 51.692 €

2.8 Nutzen und Weiterentwicklung des Ergebnis (???)

Inhalt ca 1 Seite

3 Projektablauf

Planumfang ca. 9 Seiten Davon: Dennis 3 Seiten, Stefan 3 Seiten, Gereon 3 Seiten

3.1 Teambildung und Rollenverteilung (Dennis Borutta)

ca 1 Seite

- Wie war die Teambildung für das Projekt?
 - Kurzes objektives resüme aus dem Projektstart
- Wie fand die Rollenverteilung statt?
 - Kurzes objektives resüme aus dem Projektstart

3.2 Zusammenarbeit im Projekt (Stefan Etringer)

ca 1 Seiten

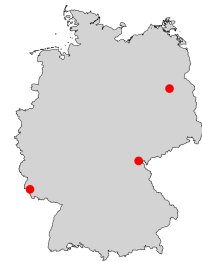
- Darstellung und Bewertung der Zusammenarbeit im Projektteam
- Objektive Beschreibung. Bewertung später.
- Gitlab, DMS, Notion

3.3 Kommunikation im Projekt (Gereon Such)

Kommunikation Gereon Such: ca 1 Seite

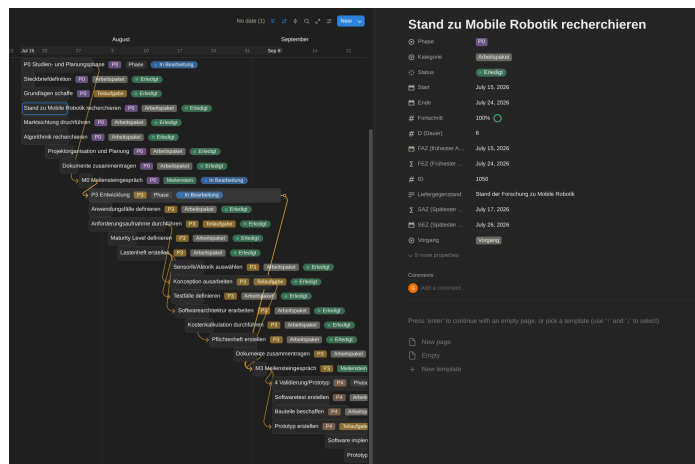
- Darstellung und Bewertung der Kommunikation im Projekt
- Objektive Beschreibung. Bewertung später.
- WhatsApp, Alfaview, Notion
- Darstellung und Bewertung der aufgetretenen Probleme bzgl. Teamarbeit und Kommunikation; Lösung dieser Probleme
- Objektive Beschreibung. Bewertung später.

Das Coasterbot Team ist bei Stichprobengröße $N = 3$ in guter Näherung über ganz Deutschland verteilt und die Kommunikation fand folglich komplett digital statt. Die Teamleitung und Kommunikation in den Meilensteingesprächen wurde durch Dennis Borutta wahrgenommen. Die Aufgaben wurden in Arbeitspakete auf- und im Team verteilt. Ein wöchentliches Meeting wurde per Videokonferenz durchgeführt, in denen jedes Teammitglied Fortschritte mitgeteilt, auf eventuelle Probleme hingewiesen sowie um eventuelle Hilfe gebeten hat. Kurze Rückmeldungen dazwischen wurden in der Woche über eine WhatsApp-Gruppe geteilt. In der Kommunikation traten keine Probleme auf.



Projektverwaltung: Notion

Das Zentrale Projektsteuerungstool war **Notion**¹. Dieses wurde zur Bestimmung von Verantwortlichkeiten, Unteraufgaben, Deadlines und Terminen genutzt, hauptsächlich gepflegt durch Dennis Borutta. Es ermöglicht neben einer wie der rechts dargestellten Timelinedarstellung auch ein kollaboratives Dokumentenmanagement, welches durch das Team insbesondere in der Anfangsphase des Projekts für Recherche und Brainstorming genutzt wurde.



¹<https://www.notion.com>

Kollaborationsplattform: Github

Das Projekt inklusive aller in L^AT_EX erstellten Dokumente, Sourcecode und anderer Entwicklungsdateien sind in einem Github² Projekt verfügbar³. Git⁴ ist ein Versionsverwaltungssystem für das dezentrale Verwalten verschiedener Versionsstände und ist für das Coasterbot Projekt gewählt worden weil das Team bereits vertraut mit diesem Tool war, es kostenlos und Cross-Platform verfügbar ist. Github ist wiederum die zentrale Plattform, an dem die Versionsstände wiederum gehostet werden.

Videokonferenz: Alfaview

Als Videokonferenzsystem für den wöchentlichen Austausch wurde auf eine Lösung der deutschen Firma Alfaview⁵ aus Karlsruhe gesetzt. Es ermöglicht neben regulärer Videokonferenz sowohl das Teilen von Dateien als auch ein Screensharing, was für den Austausch der Arbeitsstände enorm vorteilhaft war.

Kommunikation: WhatsApp

Für schnelle informelle Abstimmung und Terminabsprachen wurde der Instant-Messanging WhatsApp⁶ genutzt. Die Kommunikation erfolgte hier aufgrund der unterschiedlichen Berufsauslastung und familiärer Situation im Team asynchron.

3.4 Bewertung der Zusammenarbeit

ca 3 Seiten

- Darstellung und Bewertung der Zusammenarbeit im Projektteam
 - Subjektives Empfinden.
- Darstellung und Bewertung der Kommunikation im Projekt
 - Subjektives Empfinden.
- Darstellung und Bewertung der aufgetretenen Probleme bzgl. Teamarbeit und Kommunikation; Lösung dieser Probleme
 - Subjektives Empfinden.

²<https://github.com>

³<https://github.com/Project-CoasterBot/coasterbot>

⁴<https://git-scm.com>

⁵<https://alfaview.com>

⁶<https://whatsapp.com>

Idee: Vielleicht gibt erstellen wir eine Miniumfrage und zeigen hier die Ergebnisse.

Perspektive Gereon Such:

ca 1 Seite

Perspektive Stefan Etringer:

ca 1 Seite

Perspektive Dennis Borutta:

ca 1 Seite

3.5 Bewertung des Projektergebnis

Inhalt ca 3 Seiten

- Subjektives Empfinden. Was lief gut, was würden wir besser machen.

Perspektive Gereon Such:

ca 1 Seite

Perspektive Stefan Etringer:

ca 1 Seite

Perspektive Dennis Borutta:

ca 1 Seite